CLOUD-02: AWS Integration

> **Series:** CLOUD | **Notebook:** 2 of 8 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

This notebook covers how to set up and optimize Dynatrace's AWS integration. You will learn about IAM role-based and key-based authentication, which AWS services are supported, how to query AWS entities and metrics, and strategies for managing API costs.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`metrics.read`, `entities.read`, `ReadConfig`, `WriteConfig`
AWS Account	With IAM permissions for Dynatrace integration
Connection	Clouds app direct connection (recommended) or Environment ActiveGate (classic)
Prior Knowledge	CLOUD-01 fundamentals

1. Authentication Methods

Dynatrace supports two authentication approaches for AWS integration:

IAM Role-Based Authentication (Recommended)

IAM roles use temporary credentials via STS AssumeRole. This is the preferred approach because:
- No long-lived credentials to manage or rotate
- Follows AWS security best practices
- Supports cross-account monitoring via trust policies

Setup Steps:
1. Create an IAM role in the monitored AWS account
2. Attach the `ReadOnlyAccess` policy (or a custom least-privilege policy)
3. Add Dynatrace's AWS account as a trusted entity
4. Configure the role ARN in the **Clouds app** (or Settings > Cloud and virtualization > AWS for classic connections)

Key-Based Authentication

Uses an IAM user with access key ID and secret access key.

Aspect	IAM Role	Access Key
Credential type	Temporary (STS)	Long-lived
Rotation	Automatic	Manual (90-day recommended)
Cross-account	Trust policy	Key per account
Security risk	Low	Higher (key exposure)
Best for	Production	Development/testing

> **Important:** Regardless of method, use **least-privilege** IAM policies. `ReadOnlyAccess` is sufficient for monitoring. Never grant `AdministratorAccess`.

2. Supported AWS Services

Dynatrace monitors 80+ AWS services. Key services include:

Category	Services
Compute	EC2, Lambda, ECS, EKS, Fargate, Elastic Beanstalk
Storage	S3, EBS, EFS, Glacier
Database	RDS, DynamoDB, ElastiCache, Redshift, Aurora
Networking	ELB/ALB/NLB, CloudFront, Route 53, API Gateway, VPC
Application	SQS, SNS, Step Functions, Kinesis, EventBridge
Container	ECS, EKS, Fargate
AI/ML	SageMaker, Bedrock

Service Monitoring Granularity

Service	Entity Type	Metric Interval	Key Metrics
EC2	`dt.entity.ec2_instance`	5 min	CPU, network, disk, status checks
Lambda	`dt.entity.aws_lambda_function`	1 min	Invocations, duration, errors, throttles
RDS	`dt.entity.relational_database_service`	5 min	CPU, connections, read/write latency
S3	Custom device	1 day	Bucket size, object count, requests
ELB	`dt.entity.elastic_load_balancer`	1 min	Request count, latency, HTTP errors

3. Enabling Service Monitoring

Connection Methods

Recommended: Clouds app

1. Navigate to **Clouds app** in your Dynatrace environment
2. Select **AWS** → **Add connection**
3. Follow the guided CloudFormation deployment — this creates IAM roles, secrets, and optional log forwarding resources automatically
4. Select which AWS services to monitor

Classic: Settings UI

For Managed deployments or existing ActiveGate-based setups:

Settings > Cloud and virtualization > AWS > Add AWS connection

Service Selection Strategy

Rather than enabling all 80+ services, start with the services your applications depend on:

Tier	Services to Enable	Rationale
Tier 1 (Always)	EC2, ELB, RDS, Lambda	Core infrastructure
Tier 2 (Common)	S3, SQS, SNS, DynamoDB, ECS/EKS	Application dependencies
Tier 3 (As Needed)	CloudFront, Route 53, Kinesis, Step Functions	Edge/workflow services
Tier 4 (Selective)	SageMaker, Bedrock, IoT	Specialized workloads

Tag-Based Filtering

Use AWS tags to limit which resources are monitored:

```
# Example: Only monitor resources tagged for Dynatrace
Tag key: dynatrace-monitored
Tag value: true
```

This reduces both Dynatrace resource consumption (DPS/DDU) and AWS CloudWatch API costs.

4. Querying AWS Entities

List All EC2 Instances

```
// List all EC2 instances with their names and instance types
fetch dt.entity.ec2_instance
| fieldsKeep id, entity.name, tags, awsInstanceType, awsAvailabilityZone
| sort entity.name asc
| limit 20

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes AWS_EC2_INSTANCE
// | fieldsKeep id, name, tags, awsInstanceType, awsAvailabilityZone
// | sort name asc
// | limit 20
```

Count EC2 Instances by Instance Type

```
// Count EC2 instances grouped by instance type
fetch dt.entity.ec2_instance
| summarize instance_count = count(), by:{awsInstanceType}
| sort instance_count desc
```

List Lambda Functions

```
// List all monitored Lambda functions
fetch dt.entity.aws_lambda_function
| fieldsKeep id, entity.name, tags, awsLambdaFunctionRuntime
| sort entity.name asc
| limit 20

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes AWS_LAMBDA_FUNCTION
// | fieldsKeep id, name, tags, awsLambdaFunctionRuntime
// | sort name asc
// | limit 20
```

List RDS Instances

```
// List all monitored RDS instances
fetch dt.entity.relational_database_service
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20
```

5. AWS Metrics with DQL

EC2 CPU Usage

```
// EC2 CPU usage over the last 6 hours, top 10 busiest instances
timeseries avgCpu = avg(dt.host.cpu.usage), from:-6h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Lambda Invocation Metrics

```
// Lambda invocations over the last 6 hours by function
timeseries invocations = sum(cloud.aws.lambda.invocations), from:-6h, by:
{dt.entity.aws_lambda_function}
| fieldsAdd totalInvocations = arraySum(invocations)
| sort totalInvocations desc
| limit 10
```

Lambda Error Rate

```
// Lambda error count over the last 6 hours by function
timeseries errors = sum(cloud.aws.lambda.errors), from:-6h, by:
{dt.entity.aws_lambda_function}
| fieldsAdd totalErrors = arraySum(errors)
| filter totalErrors &gt; 0
| sort totalErrors desc
| limit 10
```

6. Metric Availability and Delays

Cloud metrics are not instantly available in Dynatrace. Understanding delays helps set appropriate alerting thresholds.

Source	Typical Delay	Notes
CloudWatch (standard)	5-10 minutes	Free tier, 5-min granularity
CloudWatch (detailed)	1-3 minutes	Additional cost, 1-min granularity
CloudWatch Metric Streams	2-3 minutes	Near real-time streaming
ActiveGate polling	5-15 minutes	Depends on number of services/metrics

Implications for Alerting

- **Don't set alert evaluation windows shorter than the metric delay** — a 1-minute evaluation window on a 5-minute metric will produce false negatives
- **Use sliding windows** — evaluate over 15-30 minutes for cloud metrics
- **Combine with OneAgent** — for sub-minute alerting, rely on OneAgent metrics rather than cloud metrics

7. Cost Optimization

AWS CloudWatch API calls are billed per request. Dynatrace cloud integration generates API calls for:
- **GetMetricData** — retrieving metric values
- **DescribeInstances** / **ListFunctions** — entity discovery
- **ListMetrics** — metric enumeration

Cost Reduction Strategies

Strategy	Impact	How
Disable unused services	High	Only enable services you actively monitor

Strategy	Impact	How
Tag-based filtering	High	Monitor only `dynatrace-monitored: true` resources
Reduce metric count	Medium	Disable individual metrics you don't need
Use Metric Streams	Medium	Streaming can be cheaper than polling at scale
Regional scoping	Medium	Only monitor regions where workloads run

Estimating CloudWatch API Costs

Approximate formula:

```
Monthly API calls = (services enabled) x (metrics per service) x (resources)
x (polls per day) x 30
Monthly cost = API calls / 1000 x $0.01
```

For example: 5 services x 10 metrics x 100 resources x 288 polls/day x 30 = ~43M calls/month = ~$430/month

> **Tip:** Review the CloudWatch billing dashboard monthly and correlate spikes with Dynatrace service additions.

8. Summary and Next Steps

Key Takeaways

- Use **IAM role-based authentication** for production AWS integrations
- Enable services in **tiers** based on business criticality, not all at once
- **Tag-based filtering** is the most effective cost optimization lever
- Account for **metric delays** (5-15 minutes) when configuring alerts
- Combine **cloud integration** with **OneAgent** for full-stack visibility

Next Steps

- **CLOUD-03: AWS EKS Monitoring** — Deep dive into Kubernetes on AWS

- **CLOUD-04: AWS Lambda & Serverless** — Serverless function monitoring
- **CLOUD-07: CloudWatch Log Ingestion** — Log forwarding from AWS
